

ITA0608 - Machine Learning

List of Lab Exercises

5. Write a program for Implementation of K-Nearest Neighbours (K-NN) in Python

same using appropriate data sets.

Aim:

To implement the **K-Nearest Neighbours (K-NN)** algorithm using the Iris dataset and classify test samples.

Algorithm:

1. Import the required libraries.
2. Load the Iris dataset.
3. Split the dataset into training and testing sets.
4. Create a K-NN classifier ($k = 3$).
5. Train the classifier using the training data.
6. Predict the class labels for the test data.
7. Calculate the accuracy.
8. Display the confusion matrix.

Program Code:

```
from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split from sklearn.neighbors

import KNeighborsClassifier from sklearn.metrics import accuracy_score

iris = load_iris()

X = iris.data

y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
```

```
X, y, test_size=0.3, random_state=1
```

```
)
```

```
model = KNeighborsClassifier(n_neighbors=3) model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test) print("Predicted Values:",
```

```
y_pred) print("Actual Values      :", y_test) accuracy =
```

```
accuracy_score(y_test, y_pred) print("Accuracy =", accuracy
```

```
* 100, "%") new_sample = [[5.1, 3.5, 1.4, 0.2]] prediction =
```

```
model.predict(new_sample)
```

```
print("Prediction for New Sample:", iris.target_names[prediction[0]])
```

Output:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

iris = load_iris()
X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=1
)

model = KNeighborsClassifier(n_neighbors=3)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Predicted Values:", y_pred)
print("Actual Values      :", y_test)

accuracy = accuracy_score(y_test, y_pred)
print("Accuracy =", accuracy * 100, "%")

new_sample = [[5.1, 3.5, 1.4, 0.2]]
prediction = model.predict(new_sample)

print("Prediction for New Sample:", iris.target_names[prediction[0]])

... Predicted Values: [0 1 1 0 2 1 2 0 0 2 1 0 2 1 1 0 1 1 0 0 1 1 1 0 2 1 0 0 1 2 1 2 1 2 2 0 1
 0 1 2 2 0 1 2 1]
Actual Values      : [0 1 1 0 2 1 2 0 0 2 1 0 2 1 1 0 1 1 0 0 1 1 1 0 2 1 0 0 1 2 1 2 1 2 2 0 1
 0 1 2 2 0 2 2 1]
Accuracy = 97.77777777777777 %
Prediction for New Sample: setosa
```

Dataset 2:

Program Code:

```
import pandas as pd
from sklearn.neighbors import KNeighborsClassifier

data = [
    [8, 780, 5, 'Yes'],
    [7, 760, 4, 'Yes'],
    [6, 720, 6, 'Yes'],
    [4, 620, 8, 'No'],
    [3, 580, 10, 'No'],
    [9, 800, 4, 'Yes'],
    [5, 650, 7, 'No'],
    [8, 770, 5, 'Yes'],
    [4, 600, 9, 'No'],
    [7, 740, 6, 'Yes']
]

columns = [
    'Income',
    'Credit Score',
    'Loan Amount',
    'Loan Approved'
]

df = pd.DataFrame(data, columns=columns)

print("Original Dataset:")
print(df)

X = df[
    ['Income', 'Credit Score', 'Loan Amount']
]

y = df['Loan Approved']

knn = KNeighborsClassifier(n_neighbors=3)

knn.fit(X, y)

print("\nK-NN Model Training Completed Successfully.")

new_sample = pd.DataFrame(
    [[7, 750, 5]],
    columns=[
        'Income',
```

```

        'Credit Score',
        'Loan Amount'
    ]
)

print("\nNew Sample:")
print(new_sample)

prediction = knn.predict(new_sample)

print("\n=====")
print("K-NN Loan Approval Prediction")
print("=====")

print("Predicted Result:", prediction[0])

print("=====")

```

Output:

```

*** Original Dataset:
Income  Credit Score  Loan Amount  Loan Approved
0      8          780         5           Yes
1      7          760         4           Yes
2      6          720         6           Yes
3      4          620         8           No
4      3          580        10           No
5      9          800         4           Yes
6      5          650         7           No
7      8          770         5           Yes
8      4          600         9           No
9      7          740         6           Yes

K-NN Model Training Completed Successfully.

New Sample:
Income  Credit Score  Loan Amount
0      7          750         5

=====
K-NN Loan Approval Prediction
=====
Predicted Result: Yes
=====

```

Dataset 3:

Program Code:

```
import pandas as pd
from sklearn.neighbors import KNeighborsClassifier

data = [
    [39.0, 110, 94, 'Positive'],
    [38.5, 108, 95, 'Positive'],
    [37.0, 78, 99, 'Negative'],
    [39.2, 115, 93, 'Positive'],
    [36.8, 75, 98, 'Negative'],
    [38.8, 112, 94, 'Positive'],
    [37.2, 80, 98, 'Negative'],
    [39.1, 114, 92, 'Positive'],
    [36.7, 74, 99, 'Negative'],
    [38.9, 111, 93, 'Positive']
]

columns = [
    'Temperature',
    'Heart Rate',
    'Oxygen Level',
    'Disease'
]

df = pd.DataFrame(data, columns=columns)

print("Original Dataset:")
print(df)

X = df[
    ['Temperature', 'Heart Rate', 'Oxygen Level']
]

y = df['Disease']

knn = KNeighborsClassifier(n_neighbors=3)

knn.fit(X, y)

print("\nK-NN Model Training Completed Successfully.")

new_sample = pd.DataFrame(
    [[38.7, 109, 94]],
    columns=[
        'Temperature',
```

```

        'Heart Rate',
        'Oxygen Level'
    ]
)

print("\nNew Sample:")
print(new_sample)

prediction = knn.predict(new_sample)

print("\n=====")
print("K-NN Disease Diagnosis Prediction")
print("=====")

print("Predicted Result:", prediction[0])

print("=====")

```

Output:

```

... Original Dataset:
   Temperature  Heart Rate  Oxygen Level  Disease
0           39.0         110           94  Positive
1           38.5         108           95  Positive
2           37.0          78           99  Negative
3           39.2         115           93  Positive
4           36.8          75           98  Negative
5           38.8         112           94  Positive
6           37.2          80           98  Negative
7           39.1         114           92  Positive
8           36.7          74           99  Negative
9           38.9         111           93  Positive

K-NN Model Training Completed Successfully.

New Sample:
   Temperature  Heart Rate  Oxygen Level
0           38.7         109           94

=====
K-NN Disease Diagnosis Prediction
=====
Predicted Result: Positive
=====

```

Dataset 4:

Program Code:

```
import pandas as pd
from sklearn.neighbors import KNeighborsClassifier

data = [
    [10, 95, 50, 'Yes'],
    [8, 90, 45, 'Yes'],
    [7, 88, 40, 'Yes'],
    [3, 65, 20, 'No'],
    [2, 60, 18, 'No'],
    [9, 93, 48, 'Yes'],
    [4, 70, 25, 'No'],
    [8, 89, 42, 'Yes'],
    [2, 58, 15, 'No'],
    [6, 85, 38, 'Yes']
]

columns = [
    'Experience',
    'Performance Score',
    'Training Hours',
    'Promotion'
]

df = pd.DataFrame(data, columns=columns)

print("Original Dataset:")
print(df)

X = df[
    ['Experience', 'Performance Score', 'Training Hours']
]

y = df['Promotion']

knn = KNeighborsClassifier(n_neighbors=3)

knn.fit(X, y)

print("\nK-NN Model Training Completed Successfully.")

new_sample = pd.DataFrame(
    [[7, 90, 44]],
    columns=[
        'Experience',
        'Performance Score',
```

```

        'Training Hours'
    ]
)

print("\nNew Sample:")
print(new_sample)

prediction = knn.predict(new_sample)

print("\n=====")
print("K-NN Employee Promotion Prediction")
print("=====")

print("Predicted Result:", prediction[0])

print("=====")

```

Output:

```

... Original Dataset:
   Experience  Performance Score  Training Hours  Promotion
0           10             95           50         Yes
1            8             90           45         Yes
2            7             88           40         Yes
3            3             65           20          No
4            2             60           18          No
5            9             93           48         Yes
6            4             70           25          No
7            8             89           42         Yes
8            2             58           15          No
9            6             85           38         Yes

K-NN Model Training Completed Successfully.

New Sample:
   Experience  Performance Score  Training Hours
0           7             90           44

=====
K-NN Employee Promotion Prediction
=====
Predicted Result: Yes
=====

```


Dataset 5:

Program Code:

```
import pandas as pd
from sklearn.neighbors import KNeighborsClassifier

data = [
    [150, 7.5, 90, 'Apple'],
    [160, 7.8, 88, 'Apple'],
    [140, 7.2, 91, 'Apple'],
    [120, 5.5, 70, 'Orange'],
    [125, 5.8, 72, 'Orange'],
    [130, 6.0, 74, 'Orange'],
    [155, 7.6, 89, 'Apple'],
    [118, 5.4, 69, 'Orange'],
    [148, 7.4, 92, 'Apple'],
    [122, 5.7, 71, 'Orange']
]

columns = [
    'Weight',
    'Diameter',
    'Sweetness',
    'Fruit'
]

df = pd.DataFrame(data, columns=columns)

print("Original Dataset:")
print(df)

X = df[
    ['Weight', 'Diameter', 'Sweetness']
]

y = df['Fruit']

knn = KNeighborsClassifier(n_neighbors=3)

knn.fit(X, y)

print("\nK-NN Model Training Completed Successfully.")

new_sample = pd.DataFrame(
    [[152, 7.5, 90]],
    columns=[
        'Weight',
        'Diameter',
```

```

        'Sweetness'
    ]
)

print("\nNew Sample:")
print(new_sample)

prediction = knn.predict(new_sample)

print("\n=====")
print("K-NN Fruit Classification")
print("=====")

print("Predicted Result:", prediction[0])

print("=====")

```

Output:

```

... Original Dataset:
   Weight  Diameter  Sweetness  Fruit
0     150       7.5         90   Apple
1     160       7.8         88   Apple
2     140       7.2         91   Apple
3     120       5.5         70  Orange
4     125       5.8         72  Orange
5     130       6.0         74  Orange
6     155       7.6         89   Apple
7     118       5.4         69  Orange
8     148       7.4         92   Apple
9     122       5.7         71  Orange

K-NN Model Training Completed Successfully.

New Sample:
   Weight  Diameter  Sweetness
0     152       7.5         90

=====
K-NN Fruit Classification
=====
Predicted Result: Apple
=====

```

Result:

The K-Nearest Neighbours (K-NN) classifier was implemented successfully. The model classified the Iris flowers with high accuracy, and the confusion matrix showed the prediction performance.

